SpendWise – Personal Finance Tracker

INTRODUCTION:

SpendWise is a versatile and user-friendly full-stack web application designed to streamline personal financial management. Suitable for tracking daily expenses, managing income, setting category-wise budgets, and analyzing spending behavior, it offers a comprehensive suite of tools — including interactive dashboards, pie charts, line charts, and CSV data export — that can be navigated through an intuitive and premium dark/light themed interface. Users can fully manage multiple financial accounts (Bank, Cash, Digital Wallet) while robust security features like JWT-based authentication and Bcrypt password hashing protect data integrity. SpendWise also features real-time budget threshold alerts that automatically notify users when their spending approaches predefined limits. With built-in data visualization tools powered by Recharts, users can generate visual spending reports and export transaction history for deeper offline analysis. Its flexibility, modern tech stack, and powerful features make it an ideal choice for students, professionals, and anyone seeking an efficient personal finance tracking solution.

SCENARIO Based Case Study:

Meet Priya, a college student juggling academics and part-time work who values financial discipline and organization in her daily life. Priya is responsible for managing her limited monthly budget, tracking expenses across multiple categories like food, transport, and entertainment, and ensuring she saves enough for upcoming semester fees.

Priya discovers SpendWise, a comprehensive web application designed to streamline personal finance management. Intrigued by its features, Priya decides to explore how SpendWise can help her take control of her spending habits.

User Registration and Authentication: Priya registers an account on SpendWise, providing her name, email, and a secure password. She logs in securely using JWT-based authentication, ensuring that her financial data remains private and protected.

Account Management: SpendWise allows Priya to create multiple financial accounts — she sets up a “Bank Account” for her savings, a “Cash” account for daily spending, and a “Digital Wallet” for online transactions. She can track balances independently for each account.

Transaction Logging: Priya uses SpendWise to record every income and expense transaction. When she receives her part-time salary, she logs it as income under the “Salary” category. When

she buys lunch, she logs it as an expense under "Food & Dining." Each transaction is timestamped and categorized automatically.

Budget Management: SpendWise allows Priya to set monthly budget limits for specific categories. She sets ₹3,000 for Food & Dining and ₹1,500 for Entertainment. The system tracks her spending against these limits with visual progress bars.

Smart Alerts: When Priya's food spending reaches 90% of her ₹3,000 budget, SpendWise triggers an automatic alert on her dashboard, warning her to slow down before exceeding her limit.

Dashboard Analytics: SpendWise provides Priya with a comprehensive dashboard featuring summary cards (Total Income, Total Expenses, Net Balance), a category-wise pie chart showing expense distribution, and a line chart tracking daily spending trends over time.

CSV Export: At the end of the month, Priya exports her complete transaction history as a CSV file for her personal records and shares it with her parents for transparency.

Priya's Experience: Thanks to SpendWise, Priya can now manage her personal finances more efficiently and effectively. She can focus on her studies while maintaining complete visibility over her spending patterns, budget adherence, and savings growth. SpendWise has become an indispensable tool in Priya's daily life, helping her achieve her financial goals with ease and confidence.

SYSTEM REQUIREMENTS:

Software Requirements:

To ensure smooth development, deployment, and usage of the SpendWise Web-Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable personal finance platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v18 or above):** Provides the runtime environment for building and managing frontend logic using Vite and React.

- **npm (v9 or above):** A package manager required to install dependencies for React and related libraries.
- **Python (v3.11 or above):** Powers the backend logic through FastAPI, handles API routing, data validation, and business logic.
- **pip:** Python package manager required to install backend dependencies (FastAPI, Motor, Pydantic, etc.).
- **React.js (v19):** A JavaScript library for building dynamic and responsive user interfaces.
- **Tailwind CSS (v4):** A utility-first CSS framework for rapid UI development with a premium aesthetic.
- **Browser:** Google Chrome / Firefox / Edge (latest version) — For rendering and testing the UI in real-time.
- **FastAPI:** A modern, high-performance Python web framework for building RESTful APIs with automatic documentation.
- **MongoDB Atlas:** A cloud-hosted NoSQL database used to store structured data related to users, transactions, budgets, accounts, and categories.
- **Postman:** Tool for testing APIs during development.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.

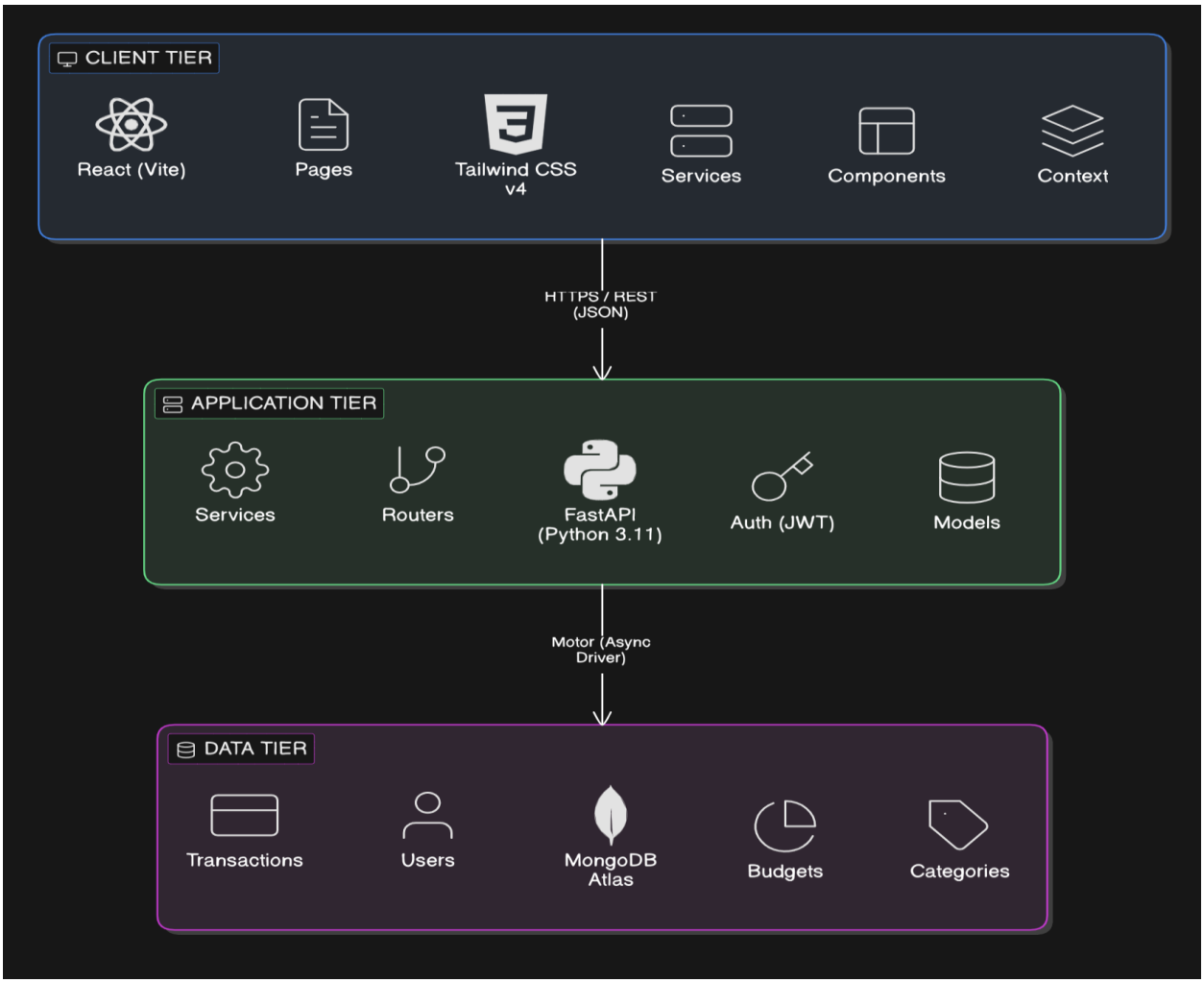
2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage:** At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.
- **Display:** 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

PROJECT ARCHITECTURE:

SpendWise follows a modern **three-tier architecture** separating the application into distinct layers:



TECHNICAL ARCHITECTURE:

In this architecture, the flow begins with a **User** interacting with the SpendWise frontend (a React application hosted on Vercel). The user performs actions such as logging in, adding transactions, or viewing their dashboard.

The SpendWise frontend then sends **HTTP requests** (via Axios) to the SpendWise backend (a FastAPI application hosted on Render). Each request includes a **JWT token** in the Authorization header for security. The backend processes the request and validates the token.

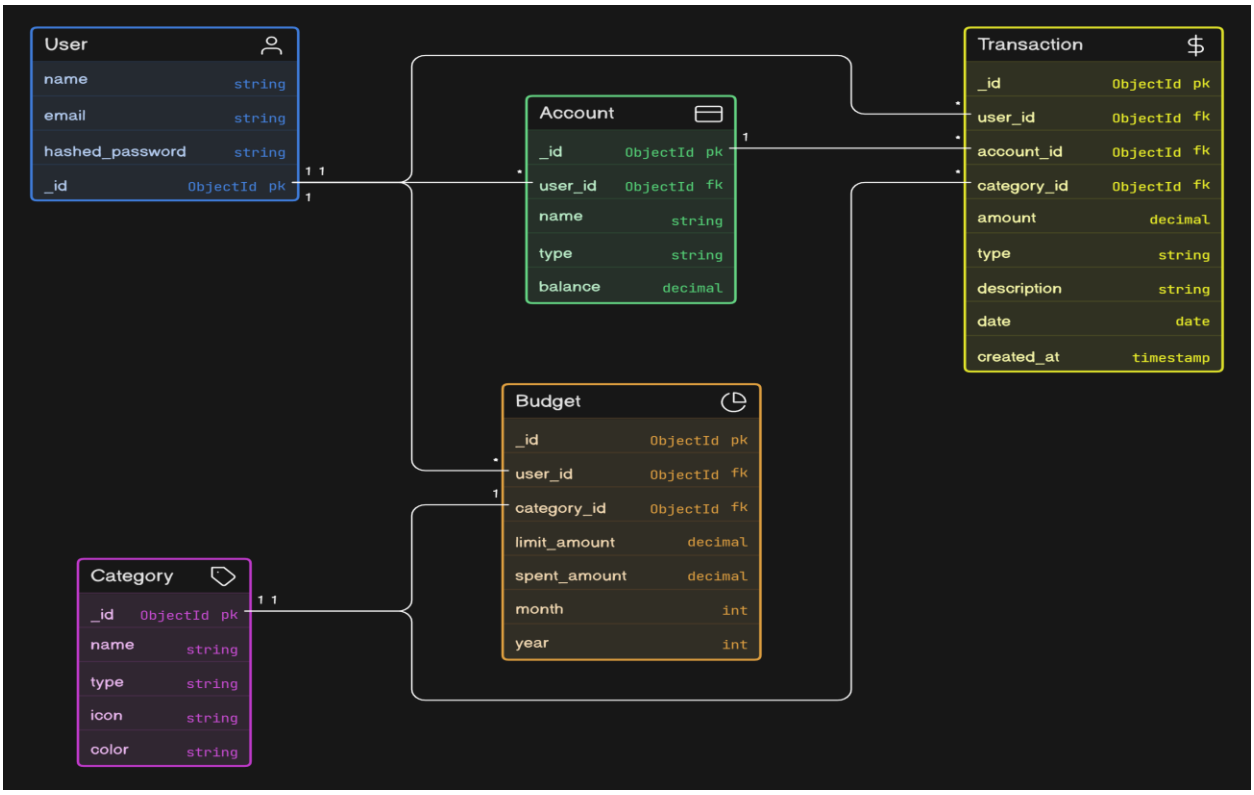
The backend communicates with **MongoDB Atlas** (a cloud-hosted NoSQL database) using the **Motor async driver** to read/write data. The database stores all user records, financial transactions, budget configurations, account details, and category definitions.

The backend applies **business logic** — such as calculating budget thresholds, aggregating dashboard statistics, and triggering budget alerts — before returning the processed response to the frontend.

Once the backend has processed the request, it sends a **JSON response** back to the React frontend. The frontend then updates its state and renders the appropriate UI components (charts, tables, alerts) to the user.

ER DIAGRAM:

The Entity-Relationship (ER) diagram for SpendWise represents the various entities and their relationships within the personal finance management system. Here is a description of the key components:



1. Entities:

- **User:** Represents registered users of the system, with attributes such as user_id, name, email, and hashed_password.
- **Transaction:** Represents financial log entries (income/expense), with attributes such as transaction_id, amount, type, description, date, and created_at.
- **Budget:** Represents monthly spending limits set by users for specific categories, with attributes such as budget_id, limit_amount, spent_amount, month, and year.
- **Account:** Represents financial accounts owned by users (Bank, Cash, Digital Wallet), with attributes such as account_id, name, type, and balance.
- **Category:** Represents predefined expense/income categories, with attributes such as category_id, name, type, icon, and color.

2. Relationships:

- **User → Transaction:** One-to-Many. A user can log multiple transactions.
- **User → Budget:** One-to-Many. A user can set multiple budgets across categories.
- **User → Account:** One-to-Many. A user can own multiple financial accounts.
- **Transaction → Category:** Many-to-One. Each transaction is classified under one category.
- **Transaction → Account:** Many-to-One. Each transaction belongs to one account.
- **Budget → Category:** One-to-One. Each budget tracks spending for one category.

3. Attributes:

Each entity has its own set of attributes that describe its properties. For example, the Transaction entity has attributes such as transaction_id, user_id, account_id, category_id, amount, type, description, date, and created_at.

4. Primary Keys:

Each entity has a primary key that uniquely identifies each record. For example, the User entity's primary key is _id (MongoDB ObjectId).

5. Foreign Keys:

Foreign keys establish relationships between entities. For example, the Transaction entity has foreign keys user_id, account_id, and category_id linking it to the respective parent entities.

Key Features:

The key features of SpendWise personal finance tracker include:

User Registration and Authentication: Users can create accounts using email and password. JWT-based authentication ensures secure access to all protected resources.

Multi-Account Management: Users can create and manage multiple financial accounts (Bank, Cash, Digital Wallet) with independent balance tracking.

Transaction Management: Users can add, edit, and delete income/expense transactions with category classification, timestamps, and optional descriptions.

Budget Management: Users can set monthly spending limits for specific categories. Progress bars visually indicate how much of each budget has been consumed.

Smart Alerts: The system automatically triggers alerts when spending reaches 90% of a defined budget limit, helping users avoid overspending.

Dashboard Analytics: A comprehensive dashboard displays Total Income, Total Expenses, and Net Balance through summary cards, a category-wise Pie Chart, and a daily spending Line Chart.

CSV Export: Users can export their complete transaction history as a downloadable CSV file for offline analysis and record-keeping.

Dark/Light Theme: A premium theme toggle allows users to switch between dark and light modes for comfortable viewing.

ROLES AND RESPONSIBILITIES:

User:

- **Registration and Account Management:** Users are responsible for creating an account, providing accurate personal information (name, email, password), and managing their profile securely.
- **Account Setup:** Users create and configure their financial accounts (Bank, Cash, Digital Wallet), setting initial balances and organizing their financial portfolio.
- **Transaction Logging:** Users are responsible for recording their income and expense transactions accurately, selecting appropriate categories, accounts, and dates.
- **Budget Configuration:** Users can set monthly spending limits for specific categories and monitor their progress through visual progress bars on the Budgets page.
- **Data Export:** Users can export their transaction history as CSV files for offline auditing, tax preparation, or sharing with financial advisors.

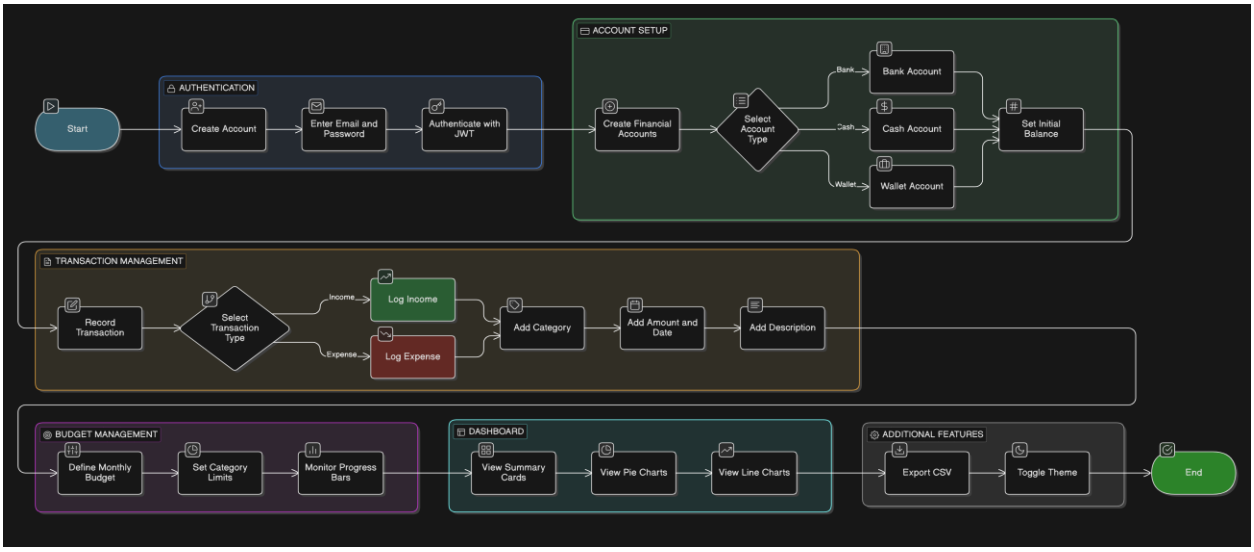
System (Automated):

- **Authentication & Security:** The system is responsible for hashing passwords using Bcrypt, generating and validating JWT tokens, and protecting all API endpoints from unauthorized access.
- **Budget Alert Monitoring:** The system automatically calculates spending against defined budgets and triggers warning alerts when thresholds are reached (90% default).
- **Dashboard Aggregation:** The system aggregates transaction data in real-time to compute total income, total expenses, net balance, category-wise breakdowns, and daily spending trends.
- **Data Integrity:** The system ensures that account balances are automatically updated when transactions are added, edited, or deleted, maintaining financial accuracy at all times.

User Flow:

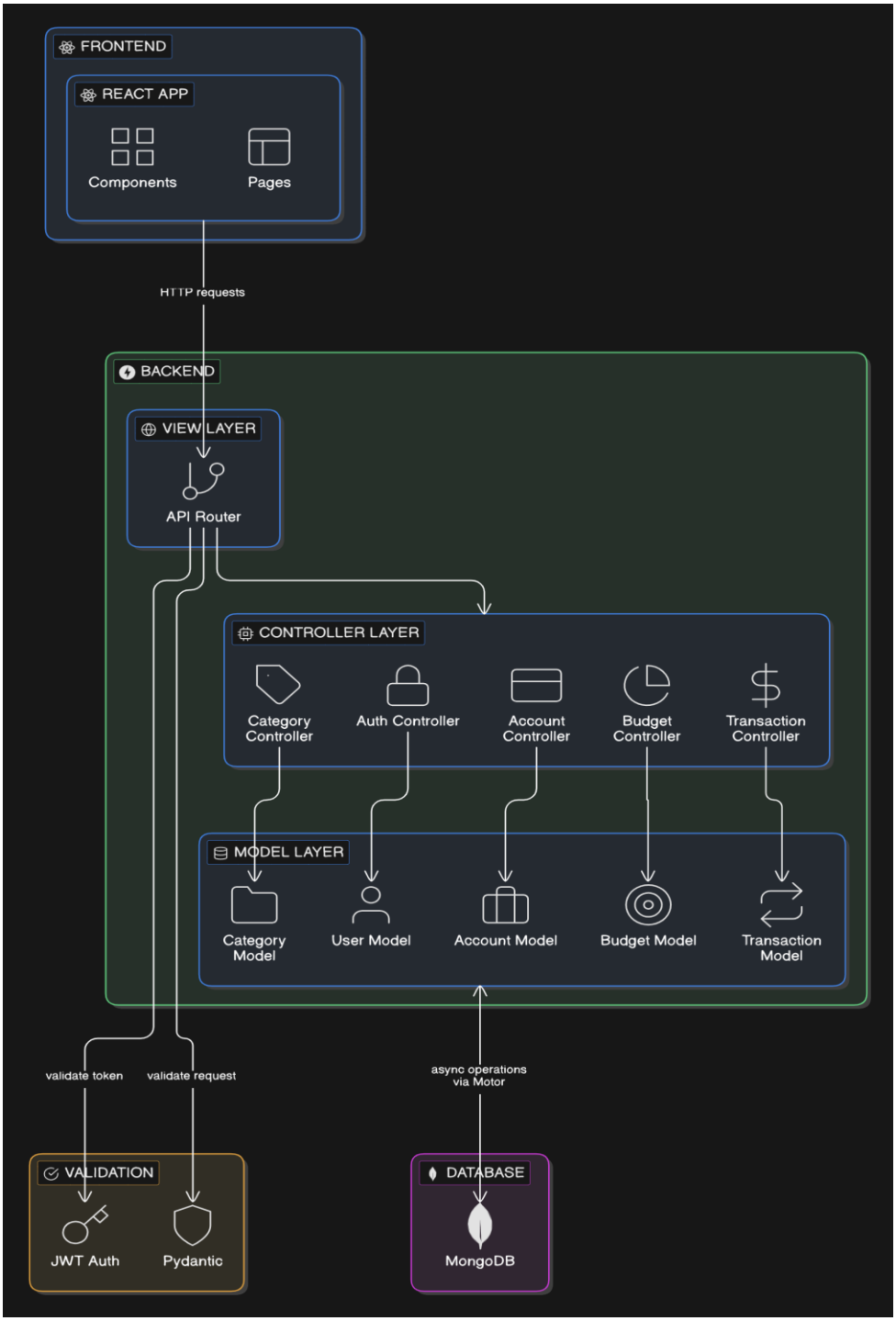
User Flow:

- **Registration & Login** – Create account with email/password, authenticate via JWT.
- **Account Setup** – Create financial accounts (Bank, Cash, Wallet) with initial balances.
- **Transaction Logging** – Record income/expense with category, amount, date, and description.
- **Budget Setting** – Define monthly limits per category, monitor via progress bars.
- **Dashboard Viewing** – View summary cards, pie charts, line charts for financial insights.
- **CSV Export** – Download complete transaction history for offline records.
- **Theme Toggle** – Switch between dark and light modes for comfortable viewing.



MVC PATTERN:

The SpendWise application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.



Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database. The models are implemented using **Pydantic** (for request/response validation) and **Motor** (for async MongoDB operations). Key models include: - User – Schema for user accounts - Transaction – Schema for income/expense records - Budget – Schema for monthly spending limits - Account – Schema for financial accounts - Category – Schema for transaction categories

Controller Layer

The Controller layer acts as an intermediary between the view (routes) and the model. In FastAPI, these are implemented as **Router functions**. They receive incoming requests, validate the JWT token, process the input (including Pydantic validation), call the appropriate database operations, and return a response to the client.

View Layer (Routing Layer)

In the context of the backend REST API, the View is implemented as the **routing layer** using FastAPI's APIRouter. Various endpoints are defined that determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

In the context of the frontend, the View is implemented as **React Pages and Components** that render the UI and handle user interactions.

Advantages of Using MVC in This Project:

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New features can be added easily by creating new routes, models, and components.
- **Reusability:** Logic in routers and models can be reused across multiple parts of the application.
- **Testing:** Each layer can be tested independently — backend via Pytest, frontend via Vitest.
- **Collaboration-Friendly:** Multiple developers can work simultaneously on different layers without conflict.

Project Setup And Configuration:

Creating project folder

1. Create a new folder with your project name (SpendWise).
2. Inside that folder create two new folders.
3. Name one as **frontend**.
4. Name another one as **backend**.
5. Now open that folder in VS Code.

Frontend setup (installing React app with Vite)

- Open the frontend folder in the terminal of VSCode.

```
npm create vite@latest . -- --template react
```

- Select React framework from the given options.
- Select JavaScript variant from the given options.
- Install all the packages:

```
npm install
```

- Install additional dependencies:

```
npm install react-router-dom axios recharts lucide-react framer-motion
npm install -D tailwindcss @tailwindcss/vite
```

- To start the React development server:

```
npm run dev
```

Backend setup (Python virtual environment)

- Open the backend folder in the terminal of VSCode.

```
python -m venv venv
```

- Activate the virtual environment:

```
.\venv\Scripts\Activate.ps1    (Windows)
source venv/bin/activate       (Mac/Linux)
```

- Install dependencies:

```
pip install -r requirements.txt
```

- Create files and folders:

- app/main.py – Application entry point
- app/database.py – MongoDB connection setup
- app/models/ – Pydantic models (user, transaction, budget, account, category)
- app/routers/ – API route handlers (auth, transactions, budgets, accounts, categories, dashboard)
- app/auth/ – JWT authentication (jwt_handler, deps)

- To start the backend server:

```
uvicorn app.main:app --reload
```

BACKEND DEVELOPMENT:

Backend Structure:

```
backend/
├── app/
│   ├── __init__.py
│   ├── main.py              → Application entry point, CORS config, router registration
│   ├── database.py          → MongoDB connection setup using Motor (async driver)
│   ├── auth/
│   │   ├── __init__.py
│   │   ├── jwt_handler.py   → JWT token creation and verification logic
│   │   └── deps.py          → Dependency injection for current user extraction
│   ├── models/
│   │   ├── __init__.py
│   │   ├── user.py          → Pydantic schemas for User registration/login/response
│   │   ├── transaction.py   → Pydantic schemas for Transaction CRUD operations
│   │   ├── budget.py        → Pydantic schemas for Budget management
│   │   ├── account.py       → Pydantic schemas for Account management
│   │   ├── category.py      → Pydantic schemas for Category definitions
│   │   └── alert.py         → Pydantic schemas for Budget alert triggers
│   ├── routers/
│   │   ├── __init__.py
│   │   ├── auth.py          → Handles user registration and login endpoints
│   │   ├── transactions.py  → Handles transaction CRUD + CSV export endpoints
│   │   ├── budgets.py       → Handles budget CRUD + threshold monitoring
│   │   ├── accounts.py      → Handles account CRUD + balance tracking
│   │   ├── categories.py    → Handles category listing and management
│   │   └── dashboard.py     → Handles dashboard aggregation (summary, charts)
│   └── services/
│       └── __init__.py
├── tests/
│   ├── conftest.py          → Pytest fixtures and test database configuration
│   └── test_app.py          → Unit tests for all API endpoints
├── requirements.txt         → Python dependencies
├── Procfile                 → Production process manager config (Gunicorn)
├── runtime.txt              → Python version pinning for Render
└── .env.example             → Template for environment variables
```

Routers:

- **auth.py** → Handles all authentication-related operations: user registration with Bcrypt password hashing, login with JWT token generation, and current user profile retrieval.
- **transactions.py** → Handles all transaction-related operations: create, read, update, delete transactions; CSV export with category/account name resolution; automatic account balance updates.
- **budgets.py** → Handles all budget-related operations: create/update monthly category budgets, track spending against limits, trigger alerts when thresholds are exceeded.
- **accounts.py** → Handles all account-related operations: create, read, update, delete financial accounts with automatic balance tracking.
- **categories.py** → Handles category management: list predefined income/expense categories, seed default categories for new users.
- **dashboard.py** → Handles dashboard data aggregation: compute total income/expenses/balance, generate category-wise pie chart data, and daily spending trend data for line charts.

Auth Middleware:

- **deps.py** → Dependency injection middleware that extracts and validates the JWT token from the Authorization header of each request. Returns the current user's ID for protected endpoints.
- **jwt_handler.py** → Contains functions for creating JWT access tokens with configurable expiry and verifying/decoding tokens to extract user identity.

Models:

- **User** (user.py):
  - UserCreate → Schema for registration (name, email, password)
  - UserLogin → Schema for login (email, password)
  - UserResponse → Schema for API responses (id, name, email)
- **Transaction** (transaction.py):
  - TransactionCreate → Schema for creating transactions (account_id, category_id, amount, type, description, date)
  - TransactionUpdate → Schema for partial updates
  - TransactionResponse → Schema for API responses
- **Budget** (budget.py):
  - BudgetCreate → Schema for creating budgets (category_id, limit_amount, month, year)

  - BudgetResponse → Schema for API responses with spending progress
- **Account** (account.py):
  - AccountCreate → Schema for creating accounts (name, type, balance)
  - AccountResponse → Schema for API responses
- **Category** (category.py):
  - CategoryCreate → Schema for category definitions (name, type, icon, color)
  - CategoryResponse → Schema for API responses

DATABASE DEVELOPMENT:

1. Configure MongoDB:

- **Install Motor** (async MongoDB driver for Python):

  pip install motor

- **Create database connection:**

  The database connection is established asynchronously using Motor's AsyncIOMotorClient. The connection URL is stored in an environment variable (MONGODB_URL) for security.

```
from motor.motor_asyncio import AsyncIOMotorClient
import os

client = None
db = None

async def connect_db():
    global client, db
    client = AsyncIOMotorClient(os.getenv("MONGODB_URL"))
    db = client.spendwise

def get_db():
    return db
```

2. Create Schemas:

The MongoDB collections store documents matching the following schemas:

1) Users Collection → Stores registered user accounts.

```
{
 "_id": ObjectId,
 "name": String,
 "email": String (unique),
 "hashed_password": String (Bcrypt hash),
```

```
  "created_at": DateTime
}
```

2) Transactions Collection → Stores all income and expense records.

```
{
  "_id": ObjectId,
  "user_id": ObjectId (ref: Users),
  "account_id": ObjectId (ref: Accounts),
  "category_id": ObjectId (ref: Categories),
  "amount": Float,
  "type": String ("income" | "expense"),
  "description": String,
  "date": DateTime,
  "created_at": DateTime
}
```

3) Budgets Collection → Stores monthly budget limits per category.

```
{
  "_id": ObjectId,
  "user_id": ObjectId (ref: Users),
  "category_id": ObjectId (ref: Categories),
  "limit_amount": Float,
  "month": Integer,
  "year": Integer,
  "created_at": DateTime
}
```

4) Accounts Collection → Stores financial accounts owned by users.

```
{
  "_id": ObjectId,
  "user_id": ObjectId (ref: Users),
  "name": String,
  "type": String ("bank" | "cash" | "wallet"),
  "balance": Float,
  "created_at": DateTime
}
```

5) Categories Collection → Stores predefined income/expense categories.

```
{
  "_id": ObjectId,
  "user_id": ObjectId (ref: Users),
  "name": String,
  "type": String ("income" | "expense"),
```

```
  "icon": String,
  "color": String
}
```

6) Alerts Collection → Stores budget threshold alert triggers.

```
{
  "_id": ObjectId,
  "user_id": ObjectId (ref: Users),
  "budget_id": ObjectId (ref: Budgets),
  "threshold_pct": Integer (default: 90),
  "triggered_at": DateTime | null
}
```

FRONT-END DEVELOPMENT:

Frontend Structure:

```
frontend/src/
├── components/
│   ├── Navbar.jsx           → Navigation bar with page links, theme toggle, user avatar
│   ├── TransactionForm.jsx  → Modal form for adding/editing transactions
│   ├── SummaryCard.jsx      → Reusable card for displaying financial metrics
│   ├── SpendingPieChart.jsx → Category-wise expense distribution pie chart
│   ├── SpendingLineChart.jsx → Daily spending trend line chart
│   ├── BudgetProgressBar.jsx → Visual budget consumption progress bar
│   ├── AlertBanner.jsx      → Dashboard alert banner for budget warnings
│   ├── ProtectedRoute.jsx   → Route guard for authenticated-only pages
│   └── ui/                  → Shared UI primitives (Button, Card, Input, etc.)
│
├── pages/
│   ├── Login.jsx            → User login page with email/password form
│   ├── Register.jsx         → User registration page
│   ├── Dashboard.jsx        → Main dashboard with summary cards, pie chart, line chart
│   ├── Transactions.jsx     → Transaction list with CRUD operations and CSV export
│   ├── Budgets.jsx          → Budget management with category-wise progress bars
│   ├── Accounts.jsx         → Financial account management page
│   └── Settings.jsx         → User settings and preferences
│
├── context/
│   ├── AuthContext.jsx      → JWT token management, login/logout state
│   └── ThemeContext.tsx     → Dark/Light theme state management
│
├── services/
│   ├── api.js               → Axios instance with base URL and JWT interceptor
│   ├── transactionService.js → API calls for transaction CRUD + export
│   ├── budgetService.js     → API calls for budget CRUD
│   ├── accountService.js    → API calls for account CRUD
│   ├── categoryService.js   → API calls for category listing
│   └── dashboardService.js  → API calls for dashboard aggregation data
│
├── App.jsx                  → Root component with React Router configuration
├── main.jsx                 → Application entry point
└── index.css                → Global styles, CSS variables, theme tokens
```

Pages:

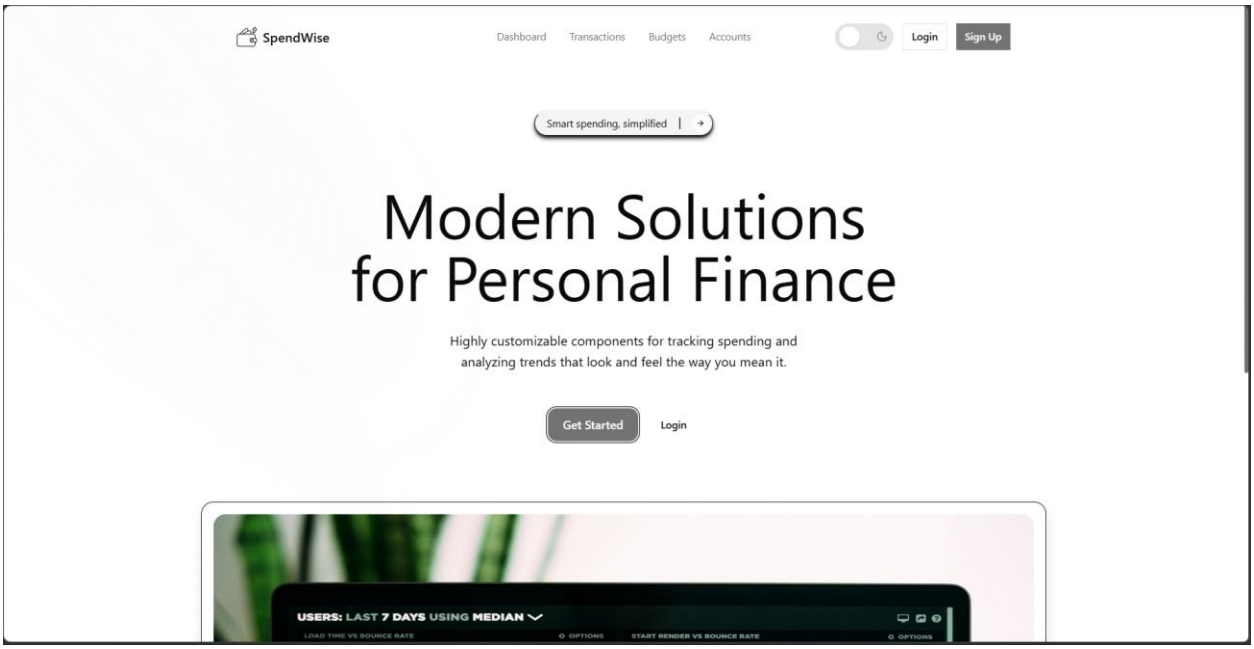
- **Login.jsx** → User login page with email and password form, JWT authentication, "Sign in with Google" option, and link to registration page.
- **Register.jsx** → User registration page with name, email, and password fields, input validation, and automatic redirect to login on success.
- **Dashboard.jsx** → Main analytics dashboard displaying Total Income, Total Expenses, and Net Balance through summary cards; a category-wise Pie Chart showing expense distribution; and a daily spending Line Chart tracking trends over time.
- **Transactions.jsx** → Full transaction management page with a data table showing Date, Type, Category, Amount, Description, and Actions (Edit/Delete). Includes a "New Transaction" modal form and a "Export CSV" button.
- **Budgets.jsx** → Budget management page where users can set monthly spending limits for specific categories. Displays visual progress bars showing current spending vs. defined limits.
- **Accounts.jsx** → Financial account management page for creating and managing Bank, Cash, and Digital Wallet accounts with balance tracking.
- **Settings.jsx** → User preferences and settings page.

Components (Reusable parts):

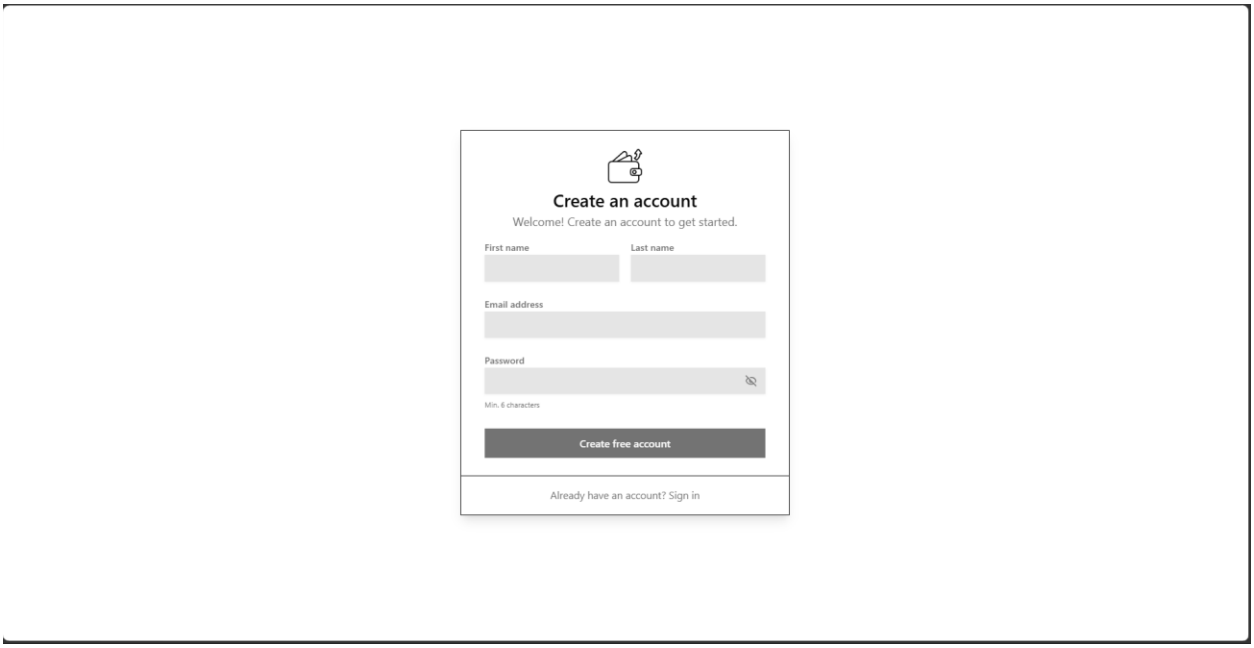
- **Navbar.jsx** → Top navigation bar with links to Dashboard, Transactions, Budgets, and Accounts; theme toggle switch; user avatar; and logout button.
- **TransactionForm.jsx** → Reusable modal form component for both creating new transactions and editing existing ones.
- **SummaryCard.jsx** → Reusable card component for displaying financial metrics with icons, labels, and formatted currency values.
- **SpendingPieChart.jsx** → Recharts-powered pie chart component showing category-wise expense breakdown.
- **SpendingLineChart.jsx** → Recharts-powered line chart component showing daily spending trends.
- **ui/** → Contains shared UI primitives: Button, Card, Input, Label, Select, Switch, Checkbox, Separator, and animation components.

Output Screenshots:

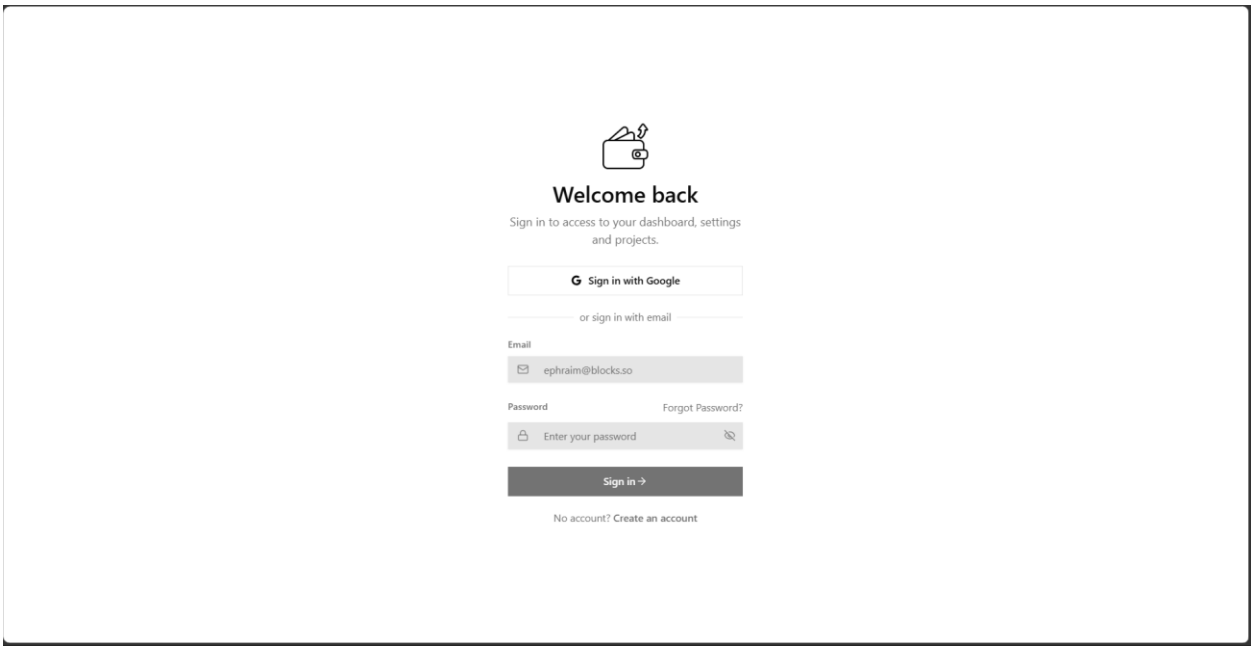
Landing Page:



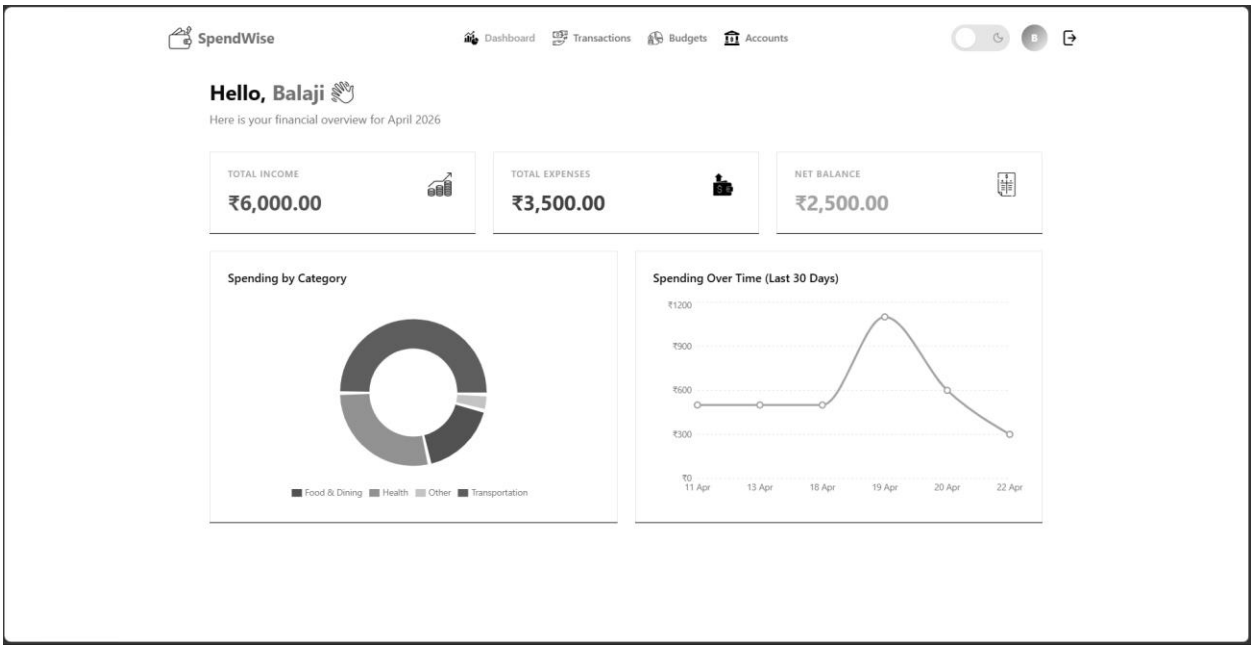
Register Page:



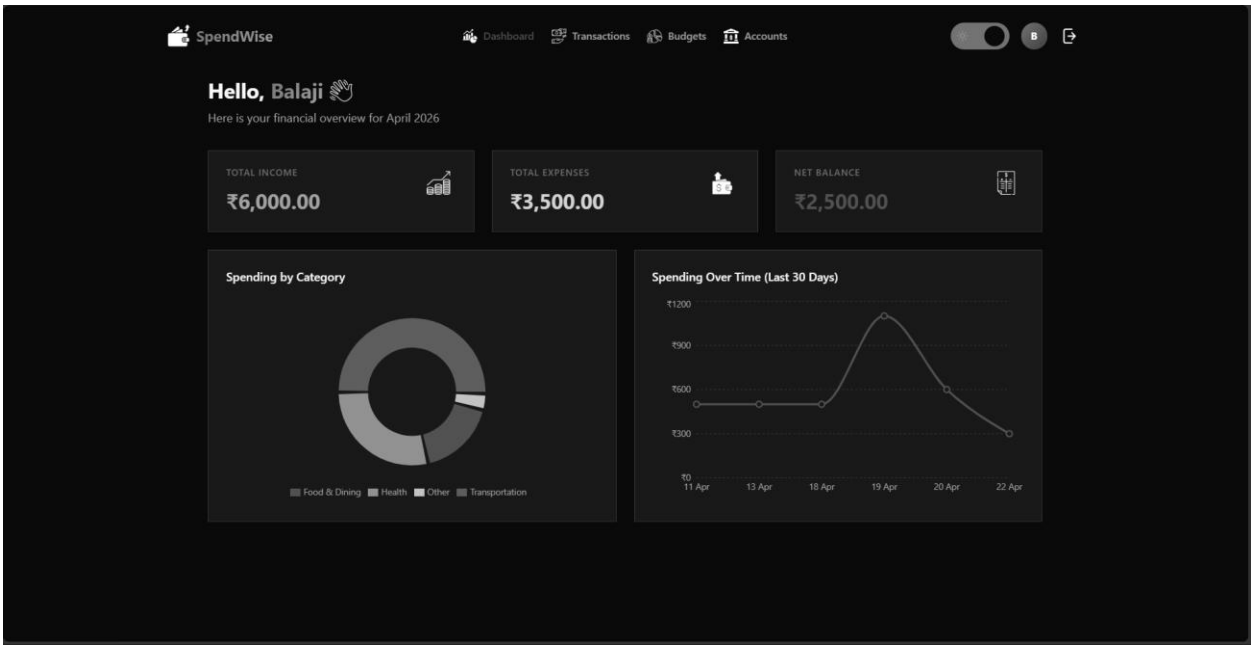
Login Page:



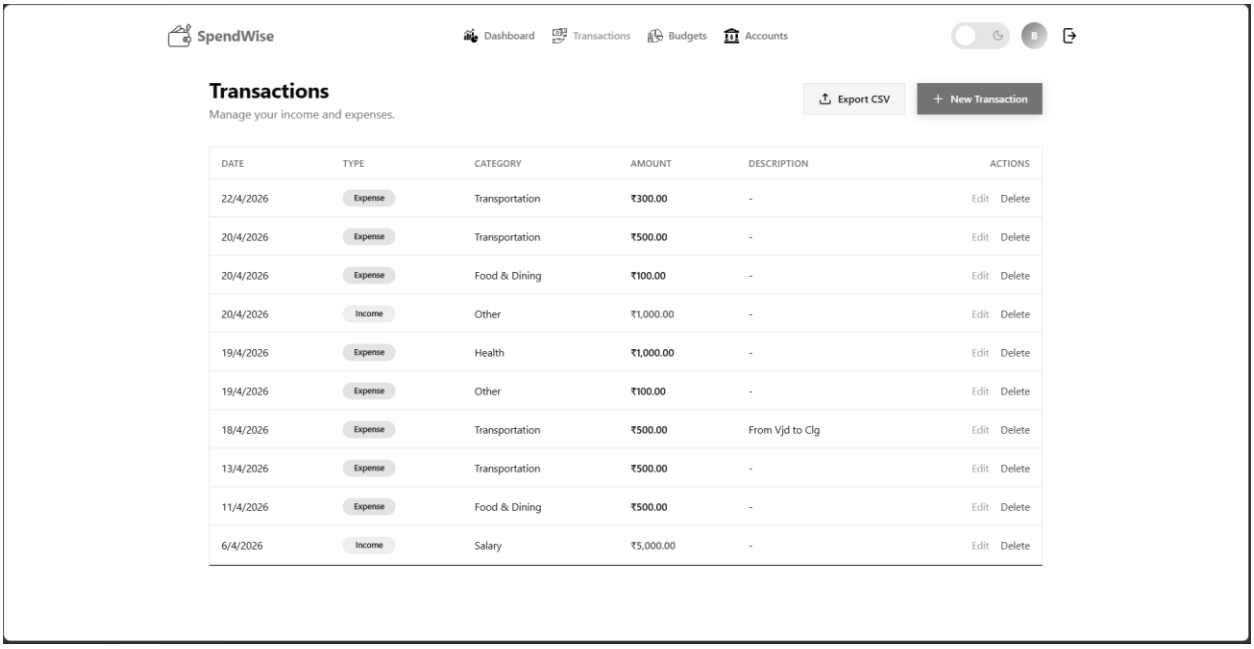
Dashboard (Light Mode):



Dashboard (Dark Mode):



Transactions Page:

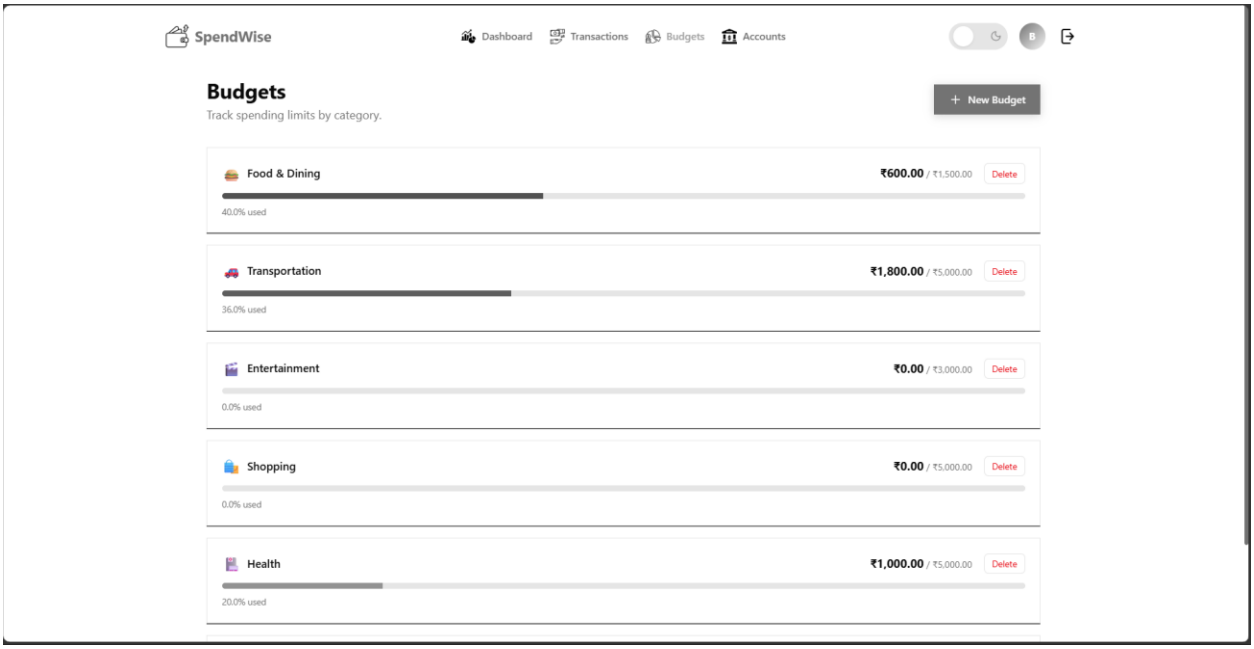


DATE	TYPE	CATEGORY	AMOUNT	DESCRIPTION	ACTIONS
22/4/2026	Expense	Transportation	₹300.00	-	Edit Delete
20/4/2026	Expense	Transportation	₹500.00	-	Edit Delete
20/4/2026	Expense	Food & Dining	₹100.00	-	Edit Delete
20/4/2026	Income	Other	₹1,000.00	-	Edit Delete
19/4/2026	Expense	Health	₹1,000.00	-	Edit Delete
19/4/2026	Expense	Other	₹100.00	-	Edit Delete
18/4/2026	Expense	Transportation	₹500.00	From Vjd to Clg	Edit Delete
13/4/2026	Expense	Transportation	₹500.00	-	Edit Delete
11/4/2026	Expense	Food & Dining	₹500.00	-	Edit Delete
6/4/2026	Income	Salary	₹5,000.00	-	Edit Delete

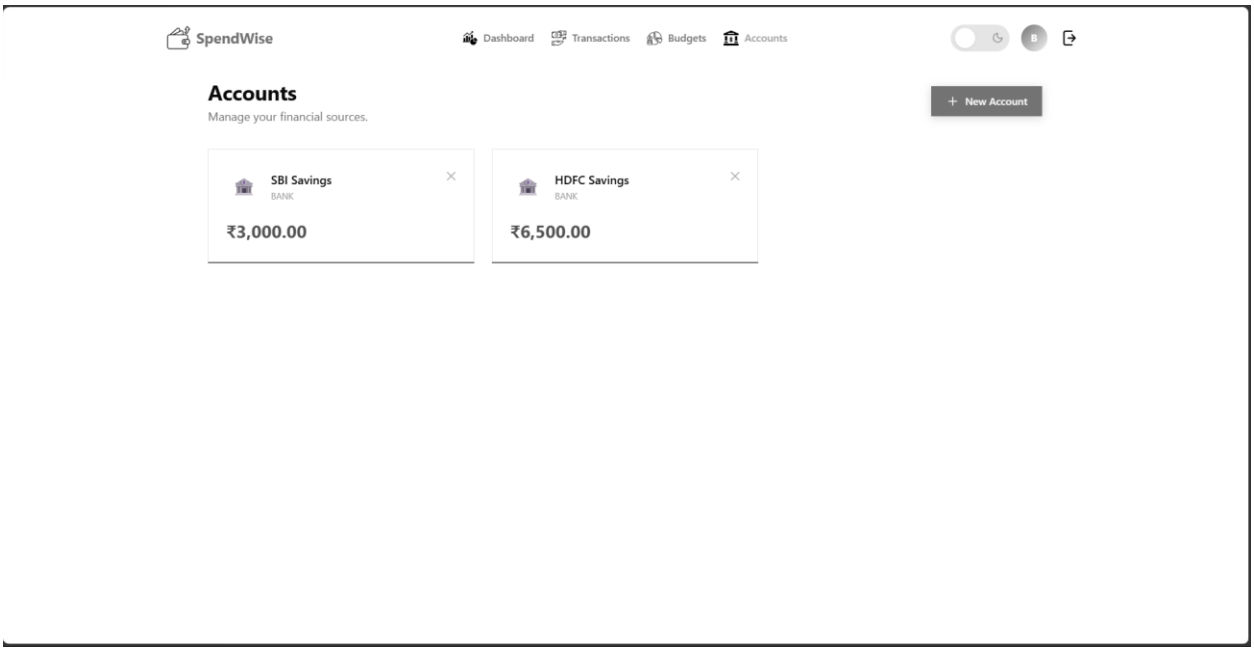
CSV Export:

	A	B	C	D	E	F
1	Date	Type	Category	Account	Amount	Description
2	22-04-2026	expense	Transportation	SBI Savings	300	
3	20-04-2026	expense	Transportation	SBI Savings	500	
4	20-04-2026	expense	Food & Dining	SBI Savings	100	
5	20-04-2026	income	Other	SBI Savings	1000	
6	19-04-2026	expense	Health	SBI Savings	1000	
7	19-04-2026	expense	Other	SBI Savings	100	
8	18-04-2026	expense	Transportation	HDFC Savings	500	From Vjd to Clg
9	13-04-2026	expense	Transportation	SBI Savings	500	
10	11-04-2026	expense	Food & Dining	SBI Savings	500	
11	06-04-2026	income	Salary	SBI Savings	5000	
12						

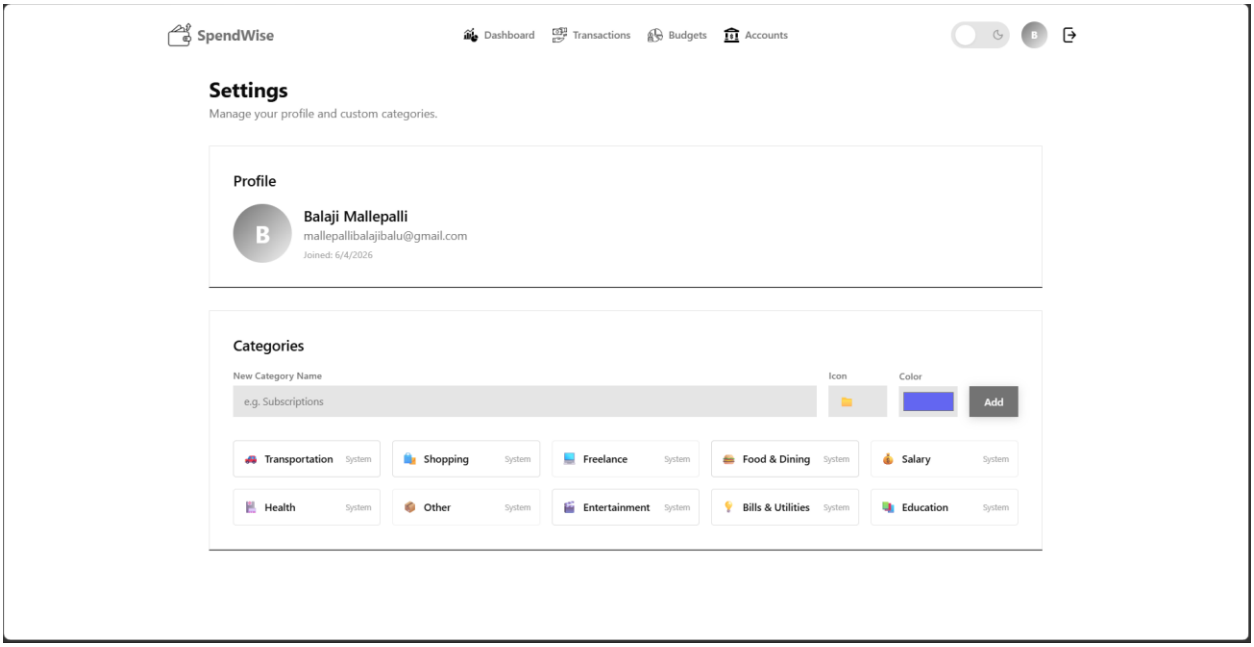
Budgets Page:



Accounts Page:



Profile/Settings Page:



Demo Video Link:

Drive Link :

https://drive.google.com/drive/folders/1mQGt6agQLdJh-3MSYyhY8EVI_WTa_AyW?usp=sharing

Code Repository Link:

GitHub:

https://github.com/balaji-mallepalli/SpendWise-Personal-Finance-Tracker

Live Deployment Links:

- Frontend (Vercel): https://spend-wise-personal-finance-tracker.vercel.app
- Backend (Render): https://spendwise-personal-finance-tracker.onrender.com